Implementation of Digital Hearing Aid System using MATLAB

BME 3112: Biomedical Signal Processing Laboratory

Submitted to:

Amit Dutta Roy

Lecturer

Department of Biomedical Engineering

Khulna University of Engineering & Technology (KUET)

Khulna -9203, Bangladesh.

Nuren Durdana Anika

Lecturer

Department of Biomedical Engineering

Khulna University of Engineering & Technology (KUET)

Khulna -9203, Bangladesh.

Submitted by: Group-8

Members:

Name-Shaira Afra Redhila

Roll-2015008

Name- Ridoy Kumar Saha

Roll-2015018

Name-Rukaiya Azad Aria

Roll-2015028

Department of Biomedical Engineering

Khulna University of Engineering &Technology

Khulna -9203, Bangladesh.

19 May,2024

Background:

Hearing aids came into existence to address the pervasive and often debilitating challenge of hearing loss, which affects people's ability to communicate, socialize, and interact with their environment. Our primary motivation behind developing hearing aids was to improve the quality of life for individuals with hearing impairments by amplifying sound and making speech more intelligible.

The earliest devices in the 17th century were ear trumpets made from materials like animal horns and metal, which simply amplified sound. In the late 19th and early 20th centuries, the advent of electricity introduced the first electronic hearing aids, which were large, cumbersome, and often required external batteries.

The 1940s saw the introduction of vacuum tube hearing aids, which were more efficient and slightly smaller. The transistor revolution of the 1950s significantly miniaturized hearing aids, making them more portable and practical. The 1960s and 1970s brought further miniaturization with the development of integrated circuits, leading to behind-the-ear and in-the-ear models.

The real game changer came in the 1980s with the advent of digital technology. The first digital hearing aids were introduced in the early 1980s but were initially bulky and expensive. As microprocessor technology advanced in the 1990s, digital hearing aids became more sophisticated, offering better sound quality and customizability. By the early 2000s, digital hearing aids had largely replaced analog models, incorporating features such as noise reduction, directional microphones, and programmable settings.

In recent years, hearing aids have become even more advanced, with features like wireless connectivity, integration with smartphones and other devices, and the use of artificial intelligence for improved sound processing and user customization. These advancements have made modern hearing aids not only more effective but also more discreet and user-friendly, significantly enhancing the quality of life for individuals with hearing loss.

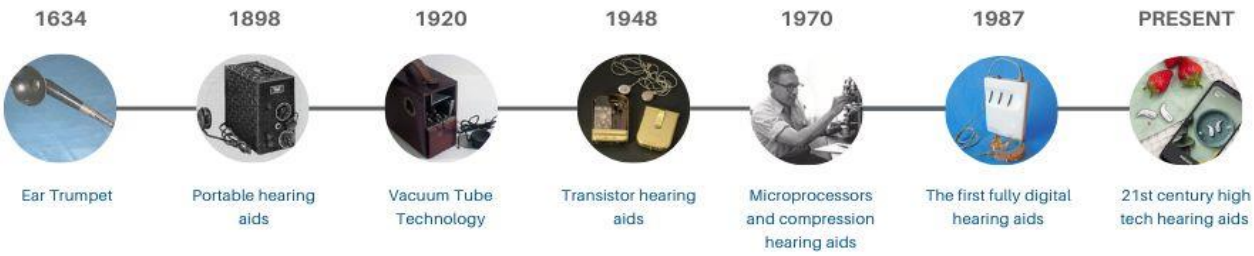


Figure no.1: A brief history of hearing aids

Objectives: Objectives of the project are as follows:

1. To record and process audio signals
2. To introduce artificial noise to the recorded audio to simulate real-world conditions
3. To apply different denoising methods to remove noise from the audio signal
4. To design and implement a frequency shaping function to amplify specific frequency ranges using FFT and IFFT
5. To create and apply a gain modification function
6. To normalize the processed audio

Introduction:

Speech is a dynamic and intricate signal. It consists of various frequencies that combine to form the sounds we perceive as words and sentences. These frequencies can be influenced by several factors such as: pitch, loudness, timbre etc. The audible frequency range for human ears is 20 Hz to 20 kHz and human hearing is most sensitive in the range of 1 kHz to 4 kHz . Hearing is measured in decibels.

Hearing loss

Hearing loss is a partial or total inability to hear sounds in one or both ears. It can affect people of all ages and can be caused by a variety of factors, including aging, exposure to loud noises, infections, and genetic conditions. Hearing loss can significantly impact communication, social interactions, and overall quality of life.

Types of Hearing Loss

Hearing loss is typically classified into three main types:

1. **Conductive Hearing Loss**: This type occurs when sound waves cannot efficiently travel through the outer ear, eardrum, or middle ear bones (ossicles). Causes include ear infections, fluid in the middle ear, earwax buildup, and structural abnormalities. Conductive hearing loss is often temporary and can sometimes be treated with medication or surgery.

2. **Sensorineural Hearing Loss:** This type results from damage to the inner ear (cochlea) or the auditory nerve pathways to the brain. Common causes include aging (presbycusis), noise exposure, head trauma, and certain medical conditions. Sensorineural hearing loss is usually permanent and is the most common type of hearing loss.

3. **Mixed Hearing Loss**: This involves a combination of conductive and sensorineural hearing loss, where there are problems in both the outer or middle ear and the inner ear or auditory nerve.

Causes of Hearing Loss

Hearing loss can be caused by various factors, including:

- **Age**: Age-related hearing loss (presbycusis) is common in older adults and typically affects both ears gradually.
- **Noise Exposure**: Prolonged exposure to loud noises can damage the hair cells in the cochlea, leading to permanent hearing loss.
- **Infections:** Ear infections, especially recurrent or chronic ones, can cause damage to the ear structures.
- **Genetic Factors**: Some forms of hearing loss are hereditary and can be present at birth or develop later in life.
- **Ototoxic Medications**: Certain medications can damage the auditory system, leading to hearing loss.
- **Trauma**: Injuries to the head or ear can result in hearing loss.

To identify the hearing loss pattern, technical researches have been conducted and conclude some plots, like spectrograms and other is audiogram.

Audiogram

An audiogram is the visual result of an audiometry hearing test administered by an audiologist or other health professional. This test measures potential hearing loss. The test uses a type of technology called an audiometer that can be connected to headphones, a speaker, or bone-conduction devices. The audiometer emits sounds that measure sound intensity and frequency.

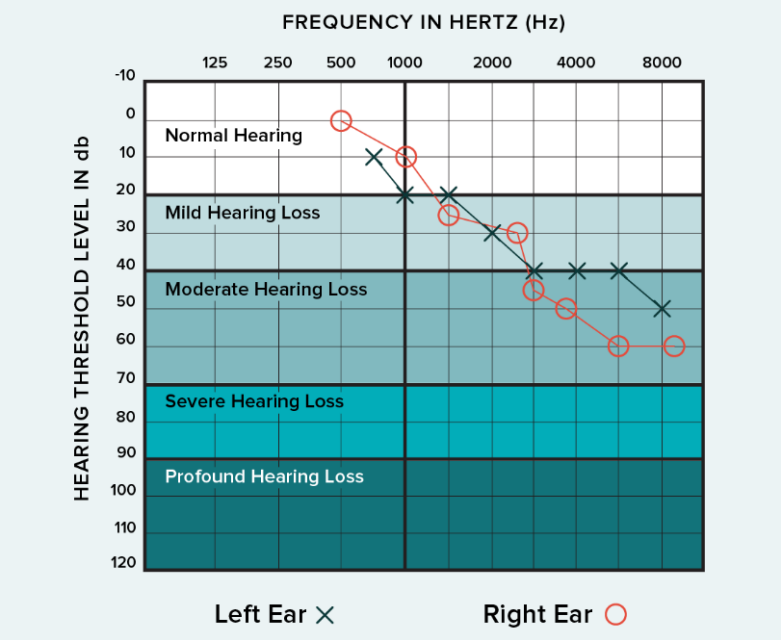


Figure no 2: Audiogram

The audiogram is a fairly simple graph:

- The Y-axis (vertical) measures the intensity, or loudness, of the sound. It's measured in decibels (Db) and ranges from -10 to 110 on the audiogram.
- The X-axis (horizontal) measures the frequency, or the pitch of the sound. The numbers run from 125 to 8,000 and measure hertz (Hz). Low-frequency sounds are the sounds of bullfrogs or thunder. High-frequency sounds can include the sounds a cricket makes, or a whistling noise.

At the end of a hearing test, hearing levels decide the degree of hearing loss. Hearing loss is measured in decibels hearing level (dB). If person can hear sounds across a range of frequencies at -10 dB-20dB it a will considered as having normal hearing. The thresholds for the different types of hearing loss are as follows:

Table no.1: Degree of hearing losses

Classification of hearing loss	**Hearing level**
Normal hearing	-10db to 26db
Mild Hearing loss	27db to 40 db
Moderate Hearing loss	40 db to 70 db
Severe Hearing loss	70 db to 90 db
Profound Hearing loss	Greater than 90 db

Hearing aid

To regain the lost hearing pattern there is a requirement of hearing aid. A hearing aid is a device that can amplify sound waves in order to help a deaf or hard-of-hearing person hear sounds more clearly. Hearing losses typically occurs non-uniformly over the audio frequency range. To compensate hearing losses, amplifiers are used in hearing aid which amplify the weak signals from the microphone uniformly over the audio range. The design of filter is another option to compensate losses, it depends upon threshold of hearing. The threshold of hearing is defined as the audio power level at which a person can no longer hear a signal at a given frequency.

Types of hearing aids:

Hearing aids come in a variety of styles, each with its unique features, advantages, and disadvantages. Understanding these can help individuals choose the best option based on their hearing needs, lifestyle, and preferences.

1. **Behind-the-Ear (BTE)**

BTE hearing aids sit behind the ear, with the main body housing the microphone, amplifier, microchip, receiver, and battery. Sound travels through thin tubing into the ear canal.

2. **Receiver-in-the-Canal (RIC)**

RIC hearing aids have the main body behind the ear, but the receiver is located in the ear canal, connected by a wired tubing. This separation helps reduce feedback.

3. **In-the-Ear (ITE)**

ITE hearing aids fit in the outer ear's bowl and are custom-made from an ear mold. They come in half-shell or full-shell models.

4. **In-the-Canal (ITC)**

ITC hearing aids fit partially in the ear canal, making them more discreet than ITE models but still larger than CIC models.

5. **Completely-in-the-Canal (CIC)**

CIC hearing aids sit deep in the ear canal and are the smallest, most discreet hearing aids available. There are also invisible-in-the-canal (IIC) aids, which fit even deeper.

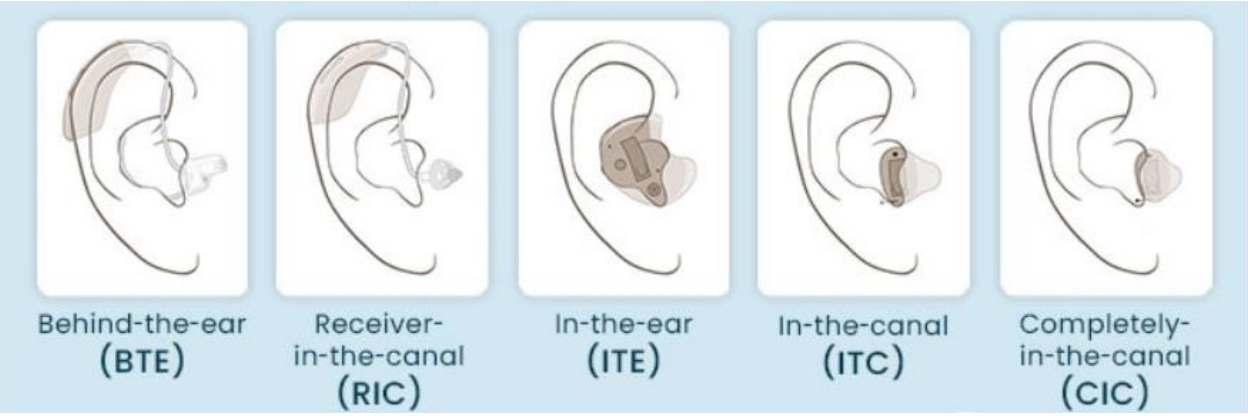


Figure no 3: Types of hearing aid

FFT(Fast Fourier Transform)

An algorithm known as a fast Fourier transform (FFT) computes a sequence's discrete Fourier transform (DFT) or its inverse (IDFT). A signal is transformed by Fourier analysis from its original domain, which is frequently time or space, to a representation in the frequency domain, and vice versa. Decomposing a series of values into components with various frequencies yields the DFT

DFT (Discrete Fourier Transform)

The Discrete Fourier transform, which is a complex-valued function of frequency, in mathematics transforms a finite sequence of evenly spaced samples of a function into a same-length sequence of evenly spaced samples of the discrete-time Fourier transform.

$$X(k)=\sum_{n=0}^{N-1} x(n) e^{\frac{-j2\pi kn}{N}}$$

where X(k) is the encode of the amplitude and phase of a sinusoidal wave with frequency $\frac{k}{N}$ cycles per time unit and x(n) are values of input signal at nth time.

Filter Design

Wavelet filter

Wavelet filtering is a powerful tool for audio signal processing due to its ability to handle both time and frequency components effectively. Wavelet filtering involves decomposing an audio signal into its constituent wavelet coefficients, applying thresholding to these coefficients to remove noise, and then reconstructing the signal. This process leverages the wavelet transform's ability to provide a multi-resolution analysis of the signal.

Parameters and Their Meanings

1. **'DenoisingMethod', 'Bayes':**

- Denoising Method: Specifies the method used for denoising the signal.
- 'Bayes': Bayesian denoising method. This method applies a Bayesian framework to estimate the clean signal. It is particularly effective in managing noise by making probabilistic assumptions about the signal and noise.

2. **'ThresholdRule', 'Soft':**

- Threshold Rule: Determines how wavelet coefficients are thresholded to reduce noise.
- `'Soft'`: Soft thresholding. This method reduces the magnitude of coefficients towards zero by a certain threshold value. It results in smoother transitions and typically better preservation of the signal structure compared to hard thresholding.

4. **'NoiseEstimate', 'LevelIndependent'**:

- Noise Estimate: Specifies how the noise level is estimated.
- 'LevelIndependent': Assumes the noise level is independent of the decomposition level, using a single noise estimate for all levels. This approach simplifies the noise estimation process and is useful when the noise characteristics are consistent across different scales.

6**. 'Wavelet', 'sym8'**:

- Wavelet Type: Specifies the type of wavelet used for the decomposition and reconstruction of the signal.
- **'sym8':** Symlet wavelet with 8 vanishing moments. Symlets are nearly symmetrical wavelets that provide good frequency localization and are well-suited for audio signal processing because they balance time and frequency resolution.

Lowpass filter

A low-pass filter is designed to pass frequencies below a specified cutoff frequency and attenuate frequencies above it. The design and specifications of a low-pass filter are crucial for its performance in signal processing applications.

Specifications of a Low-Pass Filter:

1. **Passband Frequency (Fp):** The frequency below which the frequencies are allowed to pass through the filter with minimal attenuation.

2. **Stopband Frequency (Fst)**: The frequency above which the frequencies are significantly attenuated.

3. **Passband Ripple (Ap):** The allowable variation (ripple) in the filter's gain within the passband, typically measured in decibels (dB).

4. **Stopband Attenuation (Ast):** The minimum attenuation required in the stopband, typically measured in decibels (dB).

5. **Sampling Frequency (fs):** The sampling rate of the signal to be filtered.

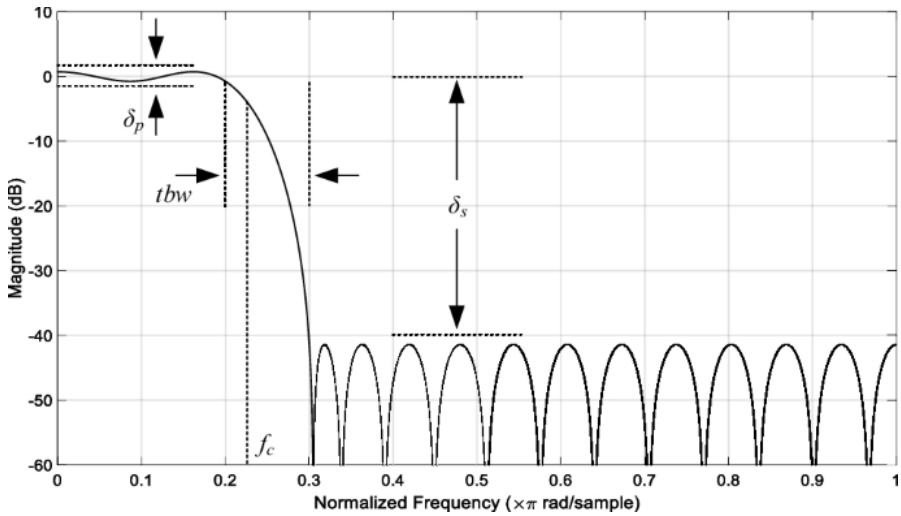


Figure no.4: Frequency response of lowpass filter

Methodology

Table 2: MATLAB functions are used to implement the code:

Function	Syntax	Description
audiorecorder	recObj = audiorecorder(Fs, nBits, nChannels)	Creates an audio recorder object with specified sampling frequency (Fs), bit depth (nBits), and number of channels (nChannels).
getaudiodata	y = getaudiodata(recObj)	Retrieves the recorded audio data from the audiorecorder object recObj.
awgn	noisy_y = awgn(y, SNR, 'measured')	Adds white Gaussian noise to the signal y to achieve the specified signal-to-noise ratio (SNR). The 'measured' option ensures the SNR is calculated based on the input signal.
fdesign.lowpass	Hlpf=fdesign.lowpass('Fp,Fst,Ap,Ast', ...)	Designs a lowpass filter object with specified passband frequency (Fp), stopband frequency (Fst), passband ripple (Ap), and stopband attenuation (Ast).
design	D = design(hlpf)	Designs the filter based on the specifications in the filter design object hlpf.
filter	xden = filter(D, noisy_y)	Applies the designed filter D to the signal noisy_y
fliplr	B = fliplr(A)	Flips the matrix A in the left-right direction, creating a mirror image.
abs	y = abs(X)	Returns the absolute value of each element in X
sound	sound(y, Fs)	Plays the audio signal y at the specified sampling frequency Fs
fft	Y = fft(x)	Computes the discrete Fourier transform (DFT) of the signal x using a fast Fourier transform (FFT) algorithm.
freqz	freqz(D)	Plots the frequency response of the filter D
wdenoise	xden = wdenoise(noisy_y, ...)	Denoises the signal noisy_y using wavelet denoising with specified parameters.

Steps:

As described in flow chart incoming speech signal is digitized to process by DSP. Signal is firstly denoised ,then passed through process of frequency shaping and amplitude limiting to keep signal amplitude within limit. The processed signal will audible to hearing impaired people.

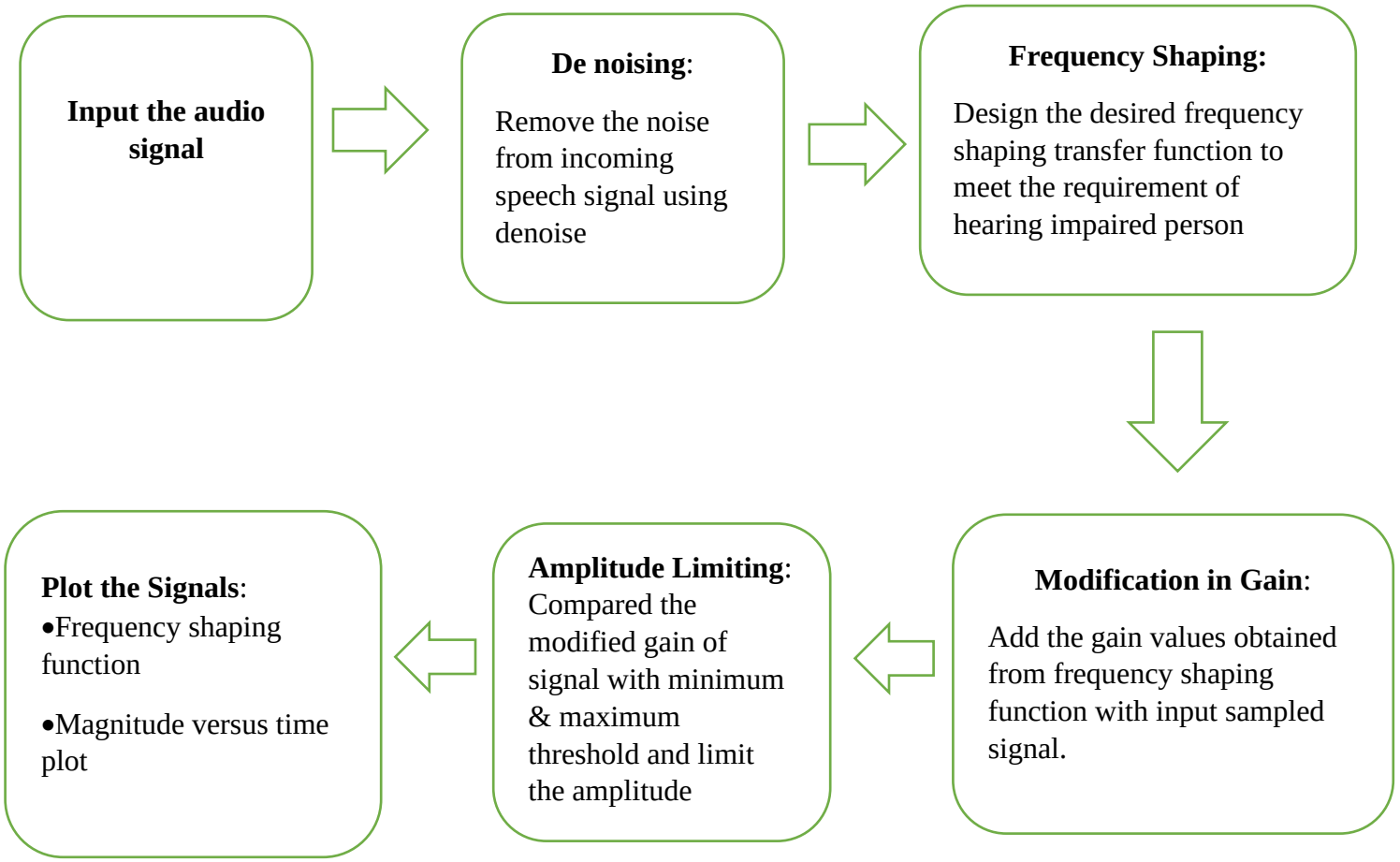


Figure 5: The Flow Chart of Hearing Aid Algorithm

Results

1. Plots of original audio signal

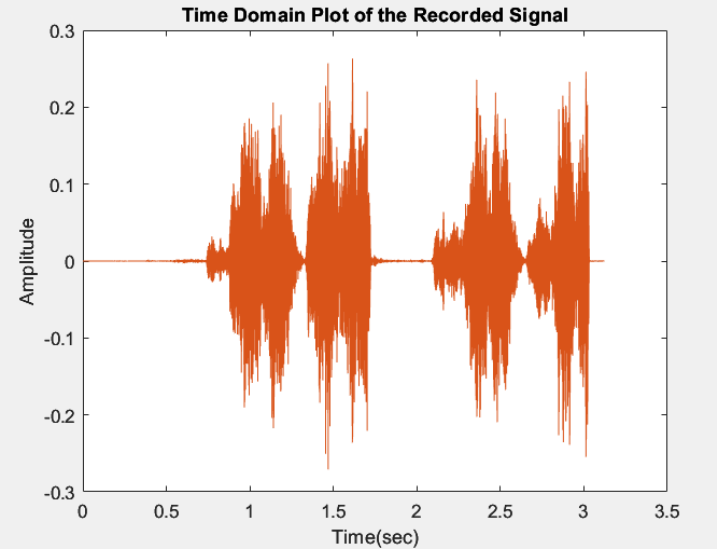


Figure no 6a:Time domain plot of recorded audio signal

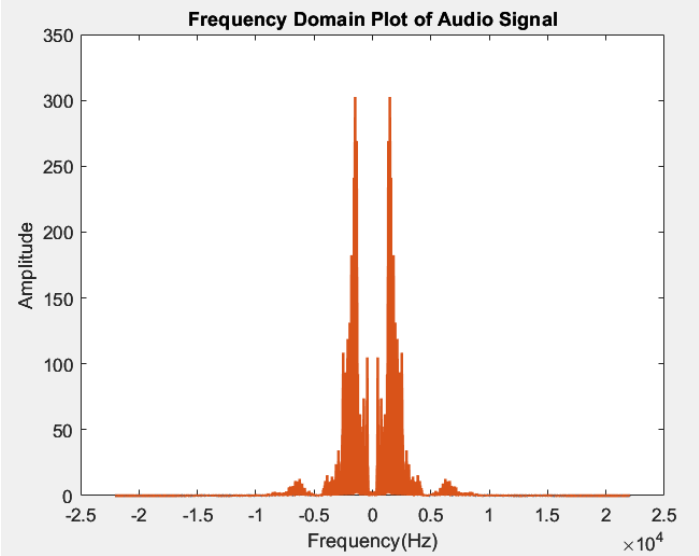


Figure no6b :Frequency domain plot of recorded audio signal

2. Plots of noise added audio signal

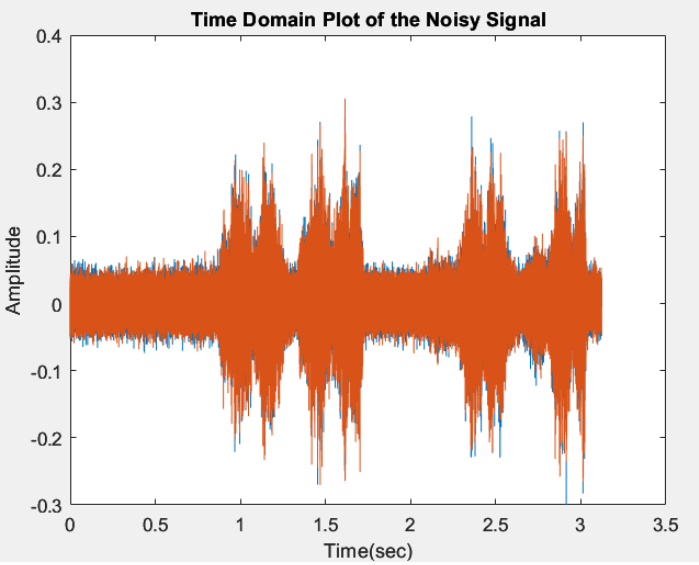


Figure no 7a:Time domain plot of noise added audio signal

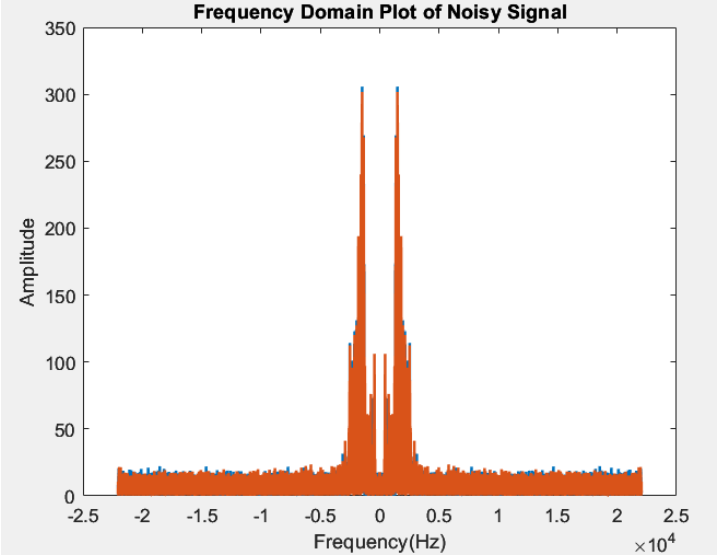


Figure no 7b :Frequency domain plot of noise added audio signal

3. Plots of denoised audio signal

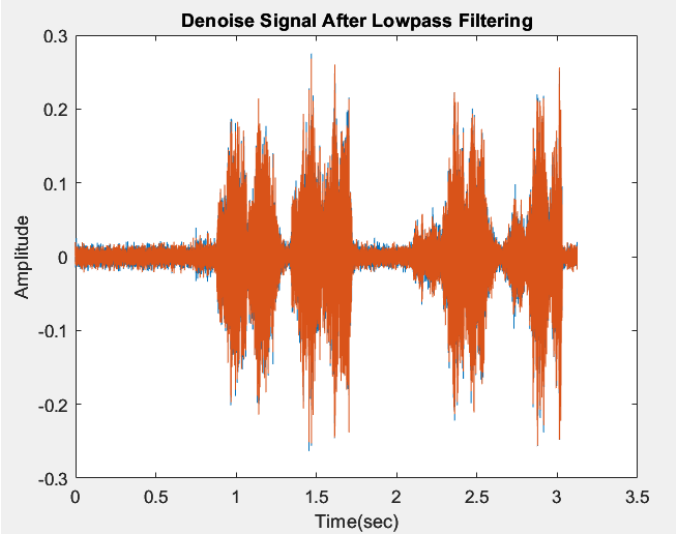


Figure no8a :Time domain plot of denoised audio signal

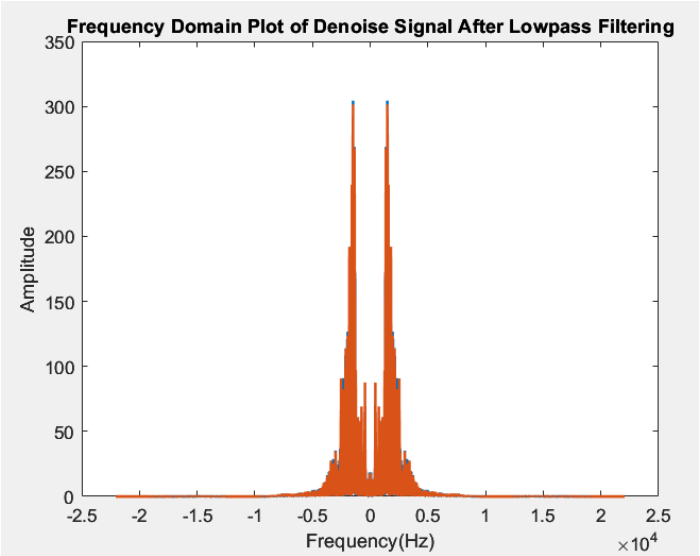


Figure no 8b:Frequency domain plot of denoised audio signal

4. Plots of gain adjusted audio signal and Gain vs Frequency curve

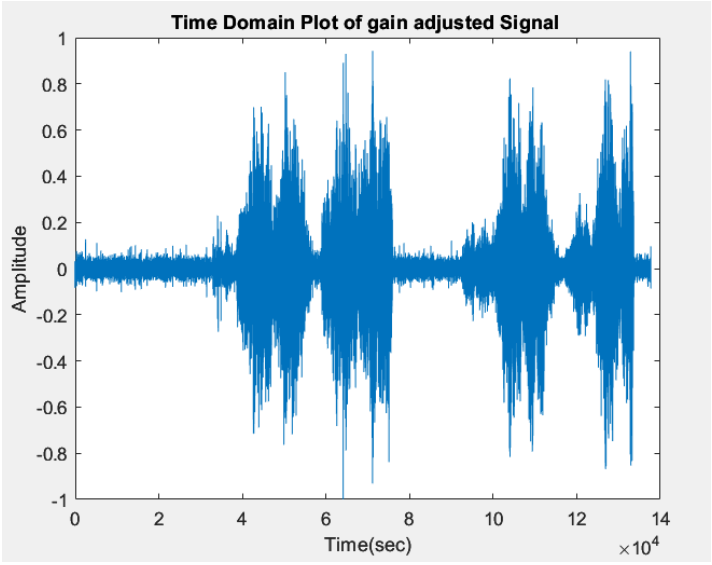


Figure no9a :Time domain plot of gain adjusted signal

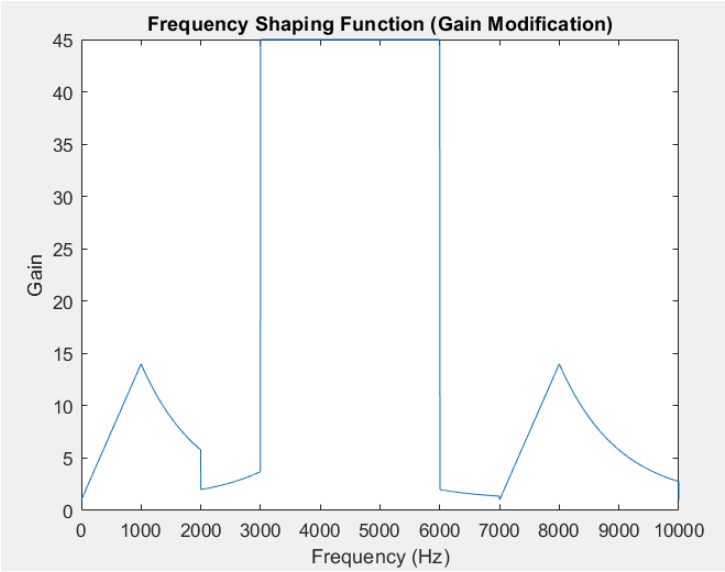


Fig no9b : Frequency shaping curve

5. Plots of amplified audio signal

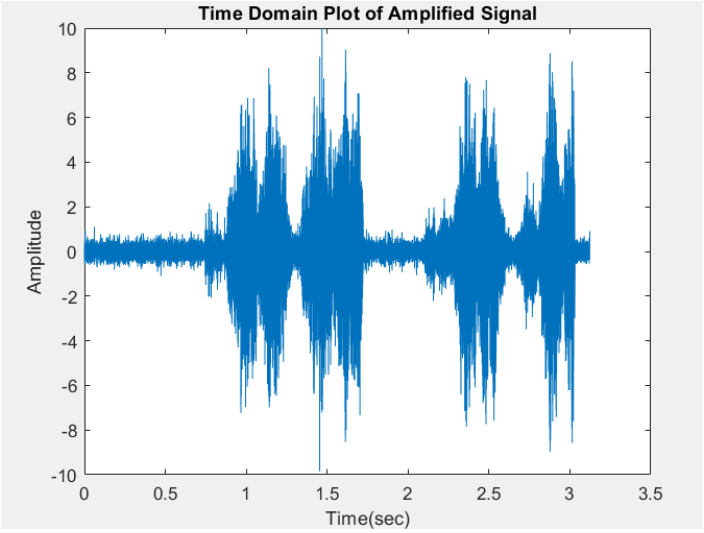


Figure no10a : Time domain plot of amplified signal

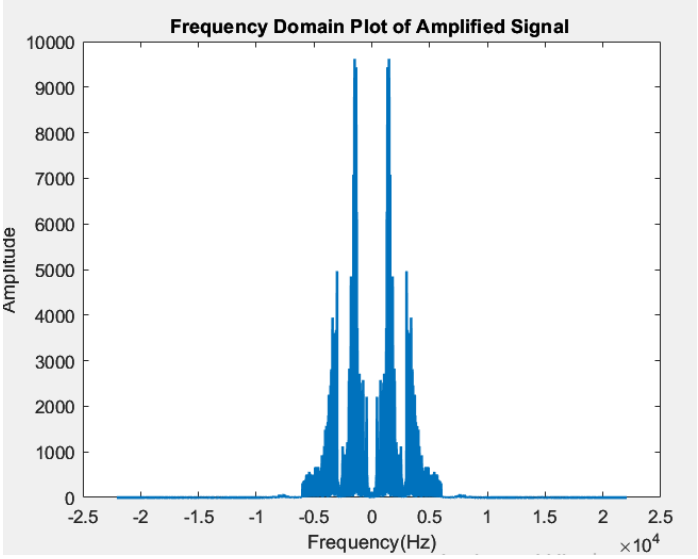


Figure no10b : Frequency domain plot of amplified signal

Analysis & Discussion

From our respective plots we have come to the following analysis:

Input vs output:

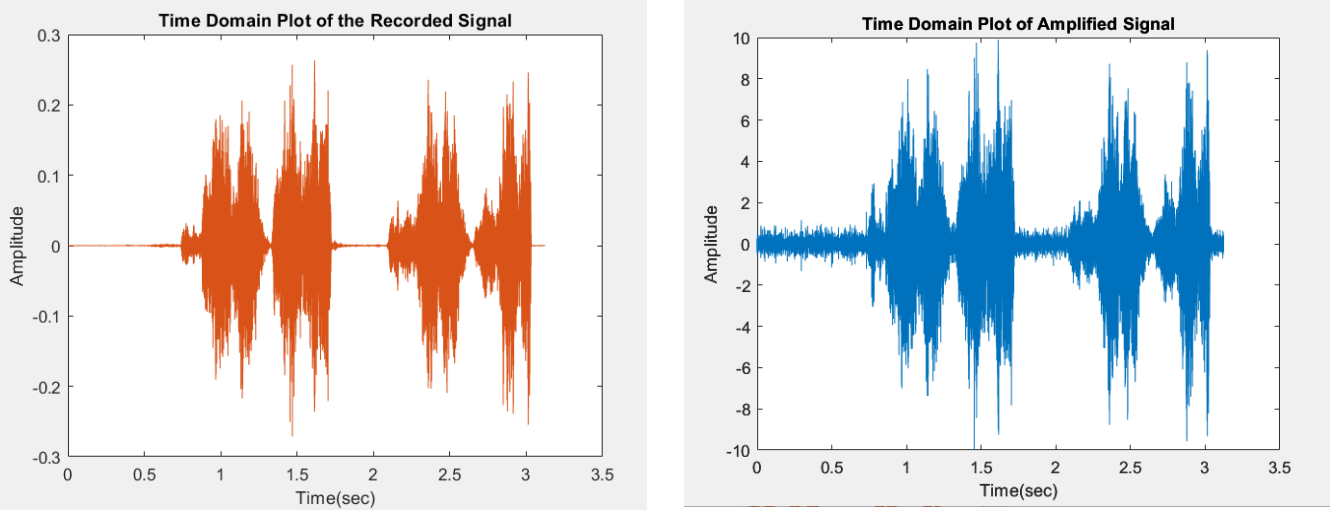
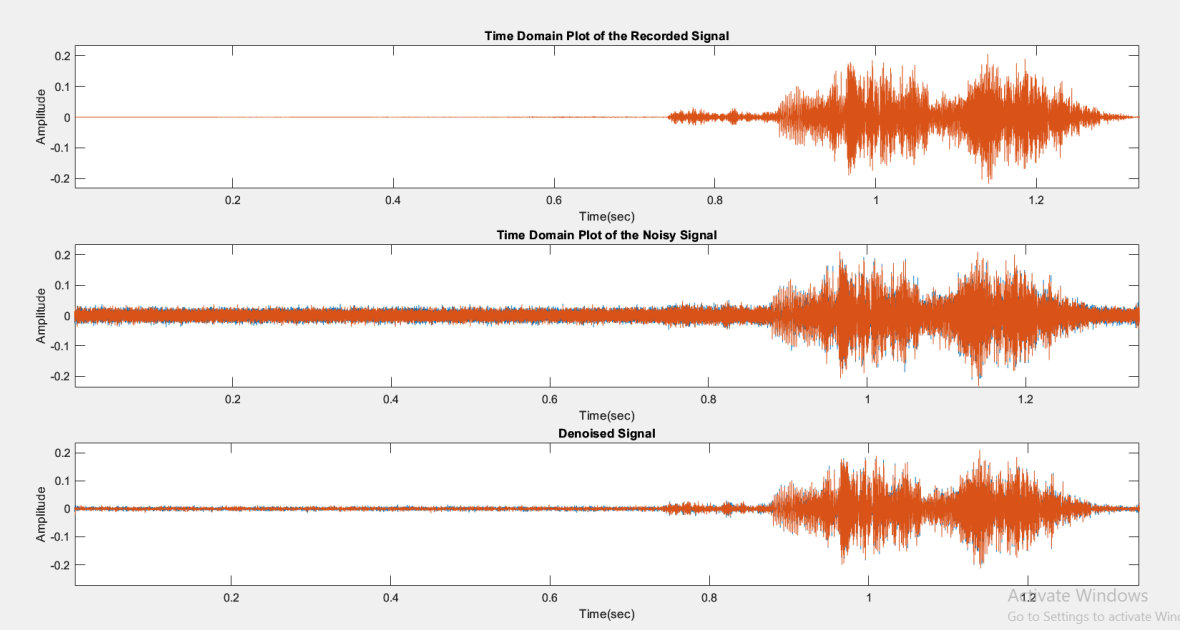


Figure no11 : Comparison of Input audio signal vs Output audio signal

Due to the presence of certain amount of noise even after filtering, noise gets amplified as well in the final audio signal output.

Denoising efficiency:

For Wavelet filtering:



For lowpass filtering with specifications:

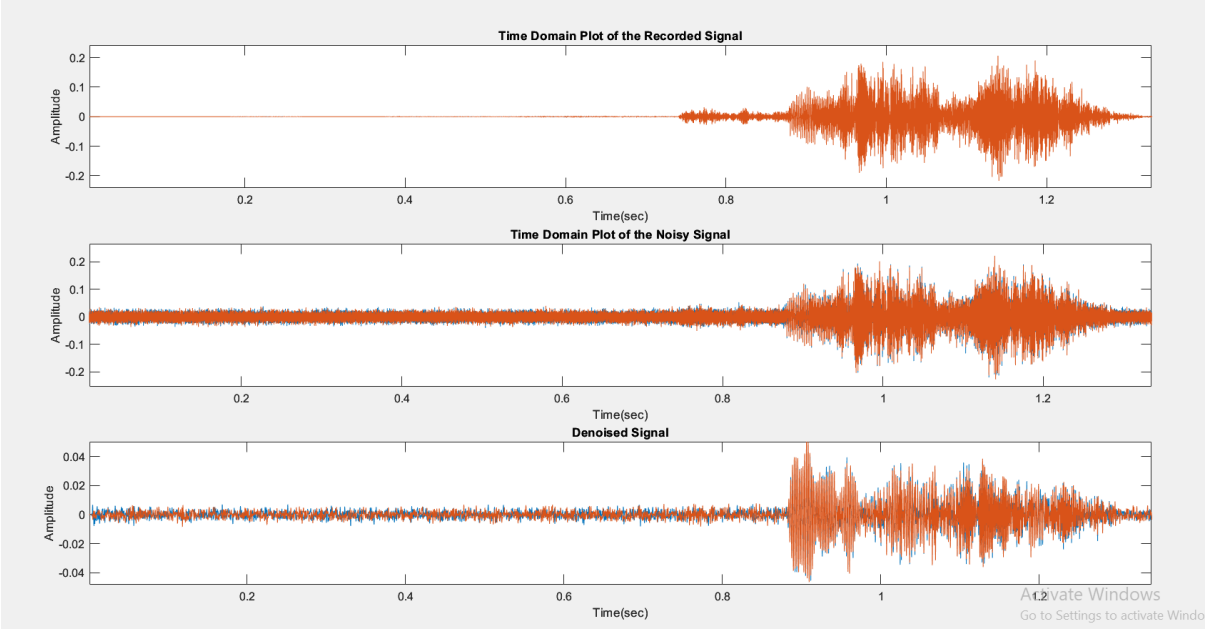


Figure no 12:Comparison of filtering/de-noising efficiency of two different filters

From the comparison we can assume that wavelet gives comparatively better denoising specifications for audio signal processing.

Error Calculation:

Code:

```
% Mean Squared Error (MSE) Function
mse = @(original, processed) mean((original - processed).^2);

% Compute MSE
mse_denoised = mse(r, x);
mse_adjusted = mse(r, shaped_signal);

% Compute Mean Original Signal Power
mean_power_original = mean(r.^2);

% Compute Percentage Error
percentage_error_denoised = (mse_denoised / mean_power_original) * 100;
percentage_error_adjusted = (mse_adjusted / mean_power_original) * 100;

% Signal-to-Noise Ratio (SNR) Calculation
snr_calc = @(original, processed) 10 * log10(sum(original.^2) / sum((original -
processed).^2));

% Compute SNR
snr_denoised = snr_calc(r, x);
snr_adjusted = snr_calc(r, shaped_signal);

% Display Results
fprintf('Percentage Error of Denoised Signal: %.4f%%\n',
percentage_error_denoised);
fprintf('SNR of Denoised Signal: %.4f dB\n', snr_denoised);
fprintf('Percentage Error of Adjusted Signal: %.4f%%\n',
percentage_error_adjusted);
fprintf('SNR of Adjusted Signal: %.4f dB\n', snr_adjusted);
```

Output:

Percentage Error of Denoised Signal: 3.5037%
SNR of Denoised Signal: 30.2283 dB
Percentage Error of Adjusted Signal: 2.49053%
SNR of Adjusted Signal: 32.5848 dB

We can thus observe that the denoising process introduced some distortion, with approximately 3.5% deviation from the original signal. With an SNR of 30.23 dB, the denoising process has effectively reduced the noise, leaving a clear signal with a high signal-to-noise ratio.

Whereas, the adjustment process has introduced slightly less distortion compared to the denoised signal, with around 2.5% deviation from the original signal. The adjusted signal maintains a high SNR of 32.58 dB, indicating that the adjustments did not significantly degrade the signal quality. Instead, they might have further improved it.

Frequency shaping:

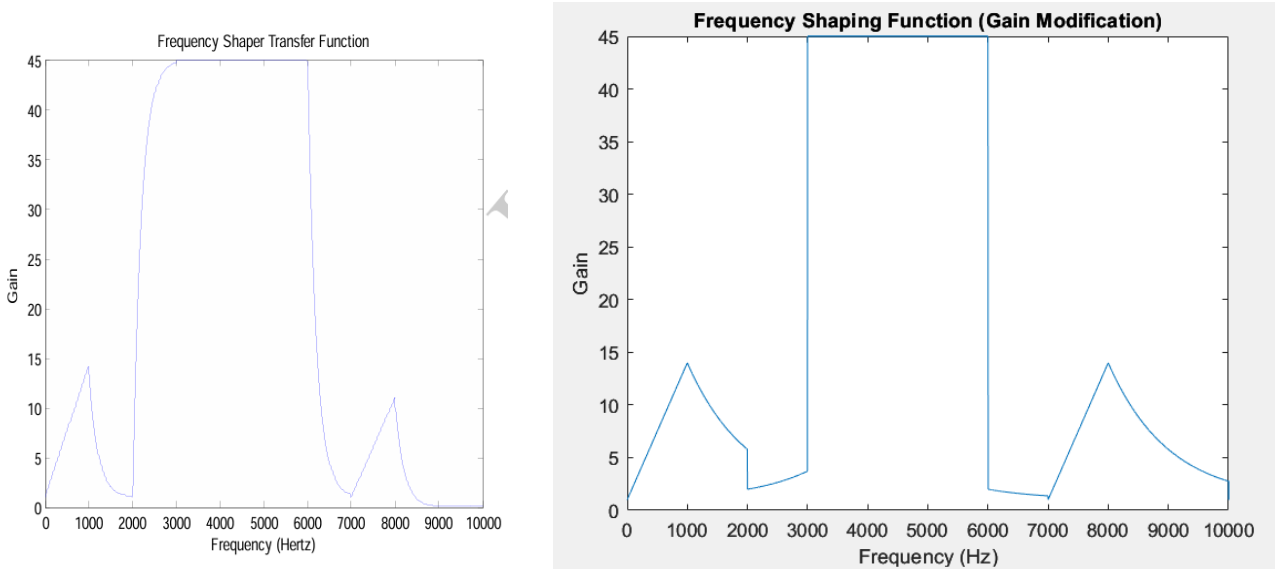


Figure no 13: Comparison of desired frequency shaping curve Vs obtained frequency shaping curve for subject who has moderate hearing loss at medium frequency range of 3000 - 6000Hz & the required gain is 45db on this frequency range.

Limitations of our digital hearing aid system:

1. **Sensitivity to Initial SNR**: The effectiveness of denoising and signal adjustment processes heavily depends on the initial SNR of the noisy signal. Higher initial SNR values may yield better results compared to lower SNR values.

2. **Subjectivity in Adjustment Parameters**: The parameters used for signal adjustment, such as gain values and frequency shaping functions, are chosen somewhat arbitrarily in the provided code. Optimizing these parameters for different audio inputs or desired outcomes may require subjective adjustments.

3. **Limited Noise Reduction Techniques** :The denoising method used (`wdenoise` function with specific parameters) may not be the most effective for all types of noise or audio signals. Different noise reduction techniques or more advanced algorithms may be required for better results in certain scenarios.

4. **Assumption of Signal Linearity :** The code assumes that the signal can be linearly amplified without causing distortion. In reality, amplifying certain frequency ranges or signals too much can lead to clipping or distortion, affecting the perceived quality of the audio.

5. **Single-Channel Processing**: The provided code operates on a single audio channel. In real-world scenarios, stereo or multi-channel audio may require additional processing techniques to maintain spatial characteristics or ensure coherence between channels.

6. **Limited Error Analysis**: The error analysis performed in the code focuses primarily on percentage error and SNR calculations. Other metrics, such as perceptual audio quality measures or frequency domain analysis, may provide deeper insights into signal fidelity and user experience.

7. **Real-time Performance Considerations**: The code does not address real-time processing requirements, which may be crucial for applications such as live audio streaming or interactive audio processing systems. Optimizing the code for real-time performance could be necessary in certain contexts.

8. **User Interface and Accessibility**: The code lacks a user-friendly interface for adjusting parameters or monitoring the processing results. For practical applications, integrating a graphical user interface (GUI) or command-line interface (CLI) may be necessary for user interaction and accessibility.

9. **Generalization to Different Audio Inputs**: The effectiveness of the denoising and adjustment processes may vary depending on the characteristics of the audio input (e.g., speech, music, environmental sounds). The code may need to be adapted or fine-tuned for different types of audio signals.

10**. Validation and Testing**: The code does not include comprehensive validation or testing procedures to assess its performance across a range of scenarios or audio inputs. Thorough testing and validation are essential to ensure the reliability and robustness of the processing algorithms.

Advanced works on digital hearing aid:

We have worked on a digital hearing aid that needs filtering components using MATLAB. Besides this, we can further add some advancement to our Project. Working on advanced hearing aids is essential to address the needs of individuals with hearing loss effectively. These advanced devices offer significant benefits such as improved sound quality, enhanced speech understanding, and better user experience. By incorporating cutting-edge technology, advanced hearing aids can adapt to different listening environments, reduce background noise, and provide personalized settings for optimal hearing assistance. Additionally, advancements in hearing aid design and features aim to enhance comfort, discretion, and connectivity, ultimately improving the quality of life for individuals with hearing impairment. Here are the advancements that has been brought to the hearing aid since 2020.

Implantable Hearing Aids: Beginning in the late 20th century, implantable hearing aids were developed and used as an alternative for individuals who were unable to tolerate conventional hearing aids. Since that time, several devices have been developed, with four currently remaining on the international market (Med-el Vibrant Soundbridge, Envoy Esteem, Ototronix MAXUM, and Cochlear Carina). This review will briefly examine the history of middle ear implant development, describe current available devices, evaluate the benefits and limits of the technology, and consider the future directions of research in the field of implantable hearing aids.

Creating Clarity in Noisy Environments by Using Deep Learning in Hearing Aids: Hearing aids continue to acquire increasingly sophisticated sound-processing features beyond basic amplification. On the one hand, these have the potential to add user benefit and allow for personalization. On the other hand, if such features are to benefit according to their potential, they require clinicians to be acquainted with both the underlying technologies and the specific fitting handles made available by the individual hearing aid manufacturers. Ensuring benefit from hearing aids in typical daily listening environments requires that the hearing aids handle sounds that interfere with communication, generically referred to as “noise.” With this aim, considerable efforts from both academia and industry have led to increasingly advanced algorithms that handle noise, typically using the principles of directional processing and postfiltering. This article provides an overview of the techniques used for noise reduction in modern hearing aids. First, classical techniques are covered as they are used in modern hearing aids. The discussion then shifts to how deep learning, a subfield of artificial intelligence, provides a radically different way of solving the noise problem. Finally, the results of several experiments are used to showcase the benefits of recent algorithmic advances in terms of signal-to-noise ratio, speech intelligibility, selective attention, and listening effort.

The Collaboration between Hearing Aid Users and Artificial Intelligence to Optimize Sound: Hearing aid gain and signal processing are based on assumptions about the average user in the average listening environment, but problems may arise when the individual hearing aid user differs from these assumptions in general or specific ways. This article describes how an

artificial intelligence (AI) mechanism that operates continuously on input from the user may alleviate such problems by using a type of machine learning known as Bayesian optimization. The basic AI mechanism is described, and studies showing its effects both in the laboratory and in the field are summarized. A crucial fact about the use of this AI is that it generates large amounts of user data that serve as input for scientific understanding as well as for the development of hearing aids and hearing care. Analyses of users' listening environments based on these data show the distribution of activities and intentions in situations where hearing is challenging. Finally, this article demonstrates how further AI-based analyses of the data can drive development.

Factors Influencing Spoken Language Outcomes in Children Following Early Use of Hearing Aids: Out of total 376 respondents 132(35.1) participants were under matric, 75(19.9%) were of matric, 68(18.1%) were of F.A, 64(17.0%) were of B.A, 37(9.8%) were of Masters qualification. Out total of 376 306(81.6%) were parents and 306(81.4%) and 66(17.6%) were with other relationship. Out of total of 376 participants, hearing aid age 3-5 years were 169(44.9%), 6-8 years were 137(39.4%), 8 above were 45(12.0%) and before 2 years were 25(6.6%). Out of total respondents 342(91%) participants were agree that the goal of hearing aid is to maximize the chances of spoken language. Out of total respondents 337(89.7%) participants were agree that early use of hearing aid enables child to have better understanding of different types of listening situations. Out of total respondents 335(89.1%) participants were agree that after use of hearing aid children maintain better interaction with normal peers through spoken language.

Improved audio experience: The improved audio experience of modern hearing aids is revolutionizing how individuals perceive sound. By incorporating cutting-edge technologies like machine learning algorithms and advanced speech recognition, these devices offer tailored and natural sound quality. Innovations such as noise suppression and cancellation techniques enhance clarity, reducing background noise interference. Companies like Tunefork and Advanced Affordable Hearing prioritize affordability without compromising on quality, ensuring a seamless auditory experience. These advancements not only address hearing loss but also elevate overall audio enjoyment, fostering inclusivity and enhancing the quality of life for individuals with hearing impairments

There are such numerous advancement on hearing aids that are already available to the general people.

Conclusion

This project successfully simulated a digital hearing aid system, enhancing audio signals for individuals with hearing impairments through advanced signal processing techniques. The historical progression of hearing aids from simple mechanical devices to sophisticated digital systems underscores the importance of improving communication and quality of life for those with hearing loss. By applying wavelet denoising, frequency shaping via FFT and IFFT, gain modification, and normalization, the project demonstrated effective noise reduction and signal enhancement. The results showed a significant reduction in noise with a 3.5% deviation from the original signal and an SNR of 30.23 dB, while gain adjustments further improved signal quality with a 2.5% deviation and an SNR of 32.58 dB. However, limitations such as sensitivity to initial SNR, parameter subjectivity, and the need for real-time processing were noted. Future work should focus on addressing these limitations, optimizing for real-time performance, and comprehensive validation to ensure robustness and practical applicability.

References:

[1] D. Hs, A. Ranjan, P. L. Pravallika, and M. Hassan, "HEARING AID FOR THE IMPAIRED USING MATLAB," vol. 7, no. 7, 2022.

[2] "How to Read Your Audiogram," Healthline. Accessed: May 19, 2024. [Online]. Available: https://www.healthline.com/health/audiogram

[3] N. Kaur and D. H. S. Ryait, "Study Of Digital Hearing Aid Using Frequency Shaping Function," *International Journal of Engineering Research*, vol. 2, no. 5, 2013.

[4] "Types of Hearing Aids & How They Work," Affordable Hearing Aids Online | MDHearing. Accessed: May 19, 2024. [Online]. Available: https://www.mdhearingaid.com/blog/types-of-hearing-aids-buyers-guide/

[5] R. M. Banakis Hartl and H. A. Jenkins, "Implantable Hearing Aids: Where are we in 2020?," Laryngoscope Investigative Otolaryngology, vol. 5, no. 6, pp. 1184–1191, 2020, doi: 10.1002/lio2.495.

[6] "Thieme E-Journals - Seminars in Hearing / Full Text." Accessed: May 19, 2024. [Online]. Available: https://www.thieme-connect.com/products/ejournals/html/10.1055/s-0041-1735134

[7] "Hearing aid benefits and limitations | Hearing Innovations — Hearing Innovations." Accessed: May 19, 2024. [Online]. Available: https://www.hearinginnovations.com.au/news/2023/10/11/hearing-aid-benefits-and-limitations

Appendix:

```
clc;
bits=16;

% Part 1: Input of sound
[r, fs] = audioread('C:\Users\Admin\Documents\3-1\bsp project\mini_dora.mp3');

t = 0:1/fs:(length(r)-1)/fs;
figure
plot(t, r);
xlabel('Time(sec)');
ylabel('Amplitude');
title('Time Domain Plot of the Recorded Signal');
disp('Playing the original Audio...');
sound(r, fs);
pause(10);

% Compute FFT of original signal
l = length(r);
f = 0:(l-1)*fs/l;
y = fft(r, l);
absy = abs(y);
f0 = (-l/2:l/2-1).*(fs/l);
y0 = fftshift(y);
absy0 = abs(y0);
figure
plot(f0, absy0, 'LineWidth', 1.5);
xlabel('Frequency(Hz)');
ylabel('Amplitude');
title('Frequency Domain Plot of Audio Signal');

% Add noise to the original signal
n = awgn(r,40);
disp('Playing noisy sound');
sound(n, fs, bits);
pause(10)

% Plot noisy signal
tn = 0:1/fs:(length(n)-1)/fs;
figure
plot(t, n);
xlabel('Time(sec)');
ylabel('Amplitude');
title('Time Domain Plot of the Noisy Signal');

% Compute FFT of noisy signal
ln = length(n);
fn = 0:(ln-1)*fs/l;
yn = fft(n, ln);
absyn = abs(yn);
f0n = (-ln/2:ln/2-1).*(fs/ln);
y0n = fftshift(yn);
absy0n = abs(y0n);
figure
```

```
plot(f0n, absy0n, 'LineWidth', 1.5);
xlabel('Frequency(Hz)');
ylabel('Amplitude');
title('Frequency Domain Plot of Noisy Signal');

% Part 2: Denoising using wavelet transform
x = wdenoise(n, 'DenoisingMethod', 'Bayes', 'ThresholdRule', 'Soft',
'NoiseEstimate', 'LevelIndependent', 8, 'Wavelet', 'sym8');
% % %part2: filtering using low pass filter
% hlpf=fdesign.lowpass('Fp,Fst,Ap,Ast',1000,1200,0.5,50,fs);
% %Lowpass filter specification
% D=design(hlpf);
% freqz(D);
% x=filter(D,n);

disp('Playing Sound After Denoising');
sound(x, fs);
pause(10)

% Plot denoised signal
figure
plot(t, x)
xlabel('Time(sec)');
ylabel('Amplitude');
title('Denoised Signal After Wavelet Denoising');

% Compute FFT of denoised signal
n1 = length(x);
y2 = fft(x, n1);
absy = abs(y2);
f00 = (-n1/2:n1/2-1).*(fs/n1);
y00 = fftshift(y2);
absy00 = abs(y00);
figure
plot(f00, absy00, 'LineWidth', 1.5);
xlabel('Frequency(Hz)');
ylabel('Amplitude');
title('Frequency Domain Plot of Denoised Signal');
figure
subplot 311
plot(t, r);
xlabel('Time(sec)');
ylabel('Amplitude');
title('Time Domain Plot of the Recorded Signal');
subplot 312
tn = 0:1/fs:(length(n)-1)/fs;
plot(t, n);
xlabel('Time(sec)');
ylabel('Amplitude');
title('Time Domain Plot of the Noisy Signal');
subplot 313
plot(t, x)
xlabel('Time(sec)');
ylabel('Amplitude');
title('Denoised Signal')
```

```
% Frequency shaping
N = length(x);
frequencies = (0:N-1) * (fs/N);
Y = fft(x);
gain = ones(size(Y));
f = 1000; % Adjust this value as needed

% Define gain based on frequency ranges
for i = 1:N/2+1
    if frequencies(i) <= 1000
        gain(i) = (frequencies(i) / 1000) * 13 + 1;
    elseif frequencies(i) <= 2000
        gain(i) = exp(-(frequencies(i) - 1000) / f) * 13 + 1;
    elseif frequencies(i) <= 3000
        gain(i) = exp((frequencies(i) - 2000) / f)  + 1;
    elseif frequencies(i) <= 6000
        gain(i) = 45;
    elseif frequencies(i) <= 7000
        gain(i) = exp(-(frequencies(i) - 6000) / f) + 1;
    elseif frequencies(i) <= 8000
        gain(i) = ((frequencies(i) - 7000) / 1000) * 13 + 1;
    elseif frequencies(i) <= 10000
        gain(i) = exp(-(frequencies(i) - 8000) / f) * 13 + 1;
    end
end

% Plot the frequency shaping function (gain modification)
figure
plot(frequencies(1:N/2+1), gain(1:N/2+1));
title('Frequency Shaping Function (Gain Modification)');
xlabel('Frequency (Hz)');
ylabel('Gain');
xlim([0 10000]);

% Apply frequency shaping
Y = Y .* gain;
shaped_signal = real(ifft(Y));
shaped_signal = shaped_signal / max(abs(shaped_signal));

disp('Playing the adjusted Audio...');
sound(shaped_signal, fs);
pause(10);

figure
plot(shaped_signal);
xlabel('Time(sec)');
ylabel('Amplitude');
title('Time Domain Plot of Gain Adjusted Signal');

% Linear amplification
amp = 10;
m = shaped_signal * amp;
disp('Playing Sound After Amplification');
sound(m, fs);
pause(10);
```

```
% Plot amplified signal
figure
plot(t, m)
xlabel('Time(sec)');
ylabel('Amplitude');
title('Time Domain Plot of Amplified Signal');

% Compute FFT of amplified signal
n2 = length(m);
y3 = fft(m, n2);
absy = abs(y3);
f000 = (-n2/2:n2/2-1).*(fs/n2);
y000 = fftshift(y3);
absy000 = abs(y000);
figure
plot(f000, absy000, 'LineWidth', 1.5);
xlabel('Frequency(Hz)');
ylabel('Amplitude');
title('Frequency Domain Plot of Amplified Signal');

% Error Calculation

% Mean Squared Error (MSE) Function
mse = @(original, processed) mean((original - processed).^2);

% Compute MSE
mse_denoised = mse(r, x);
mse_adjusted = mse(r, shaped_signal);

% Compute Mean Original Signal Power
mean_power_original = mean(r.^2);

% Compute Percentage Error
percentage_error_denoised = (mse_denoised / mean_power_original) * 100;
percentage_error_adjusted = (mse_adjusted / mean_power_original) * 100;

% Signal-to-Noise Ratio (SNR) Calculation
snr_calc = @(original, processed) 10 * log10(sum(original.^2) / sum((original -
processed).^2));

% Compute SNR
snr_denoised = snr_calc(r, x);
snr_adjusted = snr_calc(r, shaped_signal);

% Display Results
fprintf('Percentage Error of Denoised Signal: %.4f%%\n',
percentage_error_denoised);
fprintf('SNR of Denoised Signal: %.4f dB\n', snr_denoised);
fprintf('Percentage Error of Adjusted Signal: %.4f%%\n',
percentage_error_adjusted);
fprintf('SNR of Adjusted Signal: %.4f dB\n', snr_adjusted);
```